

Translated

From: e-maxx.ru

Longest increasing subsequence

We are given an array with n numbers: $a[0 \dots n-1]$. The task is to find the longest, strictly increasing, subsequence in a .

Formally we look for the longest sequence of indices $i_1, \dots, i_k$ such that

$$i_1 < i_2 < \dots < i_k, \quad a[i_1] < a[i_2] < \dots < a[i_k]$$

In this article we discuss multiple algorithms for solving this task. Also we will discuss some other problems, that can be reduced to this problem.

Solution in $O(n^2)$ with dynamic programming

Dynamic programming is a very general technique that allows to solve a huge class of problems. Here we apply the technique for our specific task.

First we will search only for the **length** of the longest increasing subsequence, and only later learn how to restore the subsequence itself.

Finding the length

To accomplish this task, we define an array $d[0 \dots n-1]$, where $d[i]$ is the length of the longest increasing subsequence that ends in the element at index i .

Example

$$a = \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\} \quad d = \{1, 1, 2, 3, 3, 1, 1, 4, 5, 2\}$$

The longest increasing subsequence that ends at index 4 is $\{3, 4, 5\}$ with a length of 3, the longest ending at index 8 is either $\{3, 4, 5, 7, 9\}$ or $\{3, 4, 6, 7, 9\}$, both having length 5, and the longest ending at index 9 is $\{0, 1\}$ having length 2.

We will compute this array gradually: first $d[0]$, then $d[1]$, and so on. After this array is computed, the answer to the problem will be the maximum value in the array d .